Manejo de Secuencias Biológicas

Alumno: Josue Samuel Philco Puma

20 de abril de 2026

1. Introducción

El manejo de secuencias biológicas implica su almacenamiento, procesamiento y análisis mediante herramientas computacionales. Estas suelen representarse en formatos estándar como **FASTA** permitiendo su manipulación eficiente en distintos programas. Y mediante estos análisis, es posible la identificación de regiones funcionales, comparar organismos, estudiar mutaciones y comprender procesos evolutivos.

2. Ejercicios

Para este laboratorio vamos a comprender lo que es la estructura de archivos biológicos para la manipulación de cadenas de **ADN/ARN** con el uso de estructuras de datos eficientes. Por lo que en este laboratorio vamos a realizar la lectura de archivos **FASTA**, mediante el identificador de toda una secuencia calcular la longitud mínima, máxima y promedio, y la proporción GC por cada secuencia.

2.1. Lectura de Archivos FASTA

Primero vamos con la parte de la lectura de los archivos **FASTA**, este archivo es utilizado para el almacenamiento de secuencias biológicas, estos archivos están compuestos por dos partes principales donde la primera línea es un encabezado que empieza por > donde esta el identificador, y la segunda parte que corresponde a la secuencia representada por las letras **A**, **T**, **C** y **G**.

Parte	**Descripción**
Encabezado (Header)	`>ENA|JBQSLH010000001|JBQSLH010000001.1 Bacillus stercoris strain MIR KLONPHBI_1, whole genome shotgun sequence.`
Secuencia	`GGAGGATTAGCTCAGCTGGGAGAGCATCTGCCTTACAAGCAGAGGGTCGGCGGTTCGAGC`

Cuadro 1: Estructura de un registro en formato FASTA

Entonces si nosotros queremos leer este archivo debemos seguir lo siguiente:

1. Leer el archivo **FASTA** correspondiente para su respectivo análisis.
2. Identificar las dos partes principales del formato: el encabezado (identificador) y la secuencia.
3. Cuando una línea comienza con el carácter >, se reconoce como un encabezado, indicando el inicio de una nueva secuencia.
4. Las líneas siguientes que no comienzan con >, van a corresponder a la secuencia biológica, por lo que se van concatenando para formar la secuencia completa.
5. Cuando se encuentra un nuevo encabezado, se procesa la secuencia previamente almacenada junto con su identificador.
6. Al finalizar la lectura del archivo, se procesa la última secuencia, ya que no habrá un nuevo encabezado que indique su cierre.

Como tal este es un proceso simple para la lectura de los archivos **FASTA** e ir identificando las secuencias que contiene. El código que lo realizaría es el siguiente:

```
void read_file_fasta(const std::string &file_fasta) {
    std::ifstream file(file_fasta);
    if (!file.is_open()) {
        std::cerr << "Error opening file: " << file_fasta << std::endl;
```

```
        return;
    }

    std::string line, sequence, header;
    while (std::getline(file, line)) {
        if (line.empty()) {
            continue;
        }

        if (line[0] == '>') {
            if (!header.empty()) {
                std::cout << "Header: " << header << std::endl;
                std::cout << "Sequence: " << sequence << std::endl;
                sequence.clear();
            }
            header = line.substr(1);
        }
        else {
            sequence += line;
        }
    }

    if (!header.empty()) {
        std::cout << "Header: " << header << std::endl;
        std::cout << "Sequence: " << sequence << std::endl;
    }

    file.close();
}
```

Entonces con este código lo que vamos a obtener es la lectura del archivo FASTA realizada, dado que estamos imprimiendo el contenido de la secuencia como se ve en la Figura 1.

Figura 1: Lectura del archivo FASTA y mostralo por consola

2.2. Estructura para el almacenamiento de cada secuencia

Ahora que podemos leer el archivo **FASTA**, debemos almacenar sus valores en una estructura, Entonces veamos que debemos guardar:

1. Debemos tener la estructura para almacenar el **identificador** y la **secuencia**.

2. Para almacenarlas podríamos utilizar vectores e ir guardando cada identificador con su secuencia.

3. La función para leer el archivo **FASTA** va a seguir siendo útil, solo que cambiariamos para que vaya alamcenando todo en el vector.

Entonces, con esto en mente debemos modificar la función para que almacene su **identificador** y **secuencia**.

```cpp
struct FastaRecordStructure {
    std::string identifier;
    std::string sequence;
};

void read_file_fasta(const std::string &file_fasta, std::vector<
    FastaRecordStructure> &records) {
    std::ifstream file(file_fasta);
    if (!file.is_open()) {
        std::cerr << "Error opening file: " << file_fasta << std::endl;
        return;
    }

    std::string line, sequence, header;
    while (std::getline(file, line)) {
        if (line.empty()) {
            continue;
        }

        if (line[0] == '>') {
            if (!header.empty()) {
                records.push_back({header, sequence});
                sequence.clear();
            }
            header = line.substr(1);
        }
        else {
            sequence += line;
        }
    }

    if (!header.empty()) {
        records.push_back({header, sequence});
    }

    file.close();
}
```

Ahora con este código podremos almacenar el **identificador** y la **secuencia** de todo el archivo **FASTA**. También lo podemos mostrar por la terminal pero para comprobar que está imprimiendo las secuencias lo reduciremos a mostrar solo 100 primeras letras de la secuencia como se ve en la Figura 2.

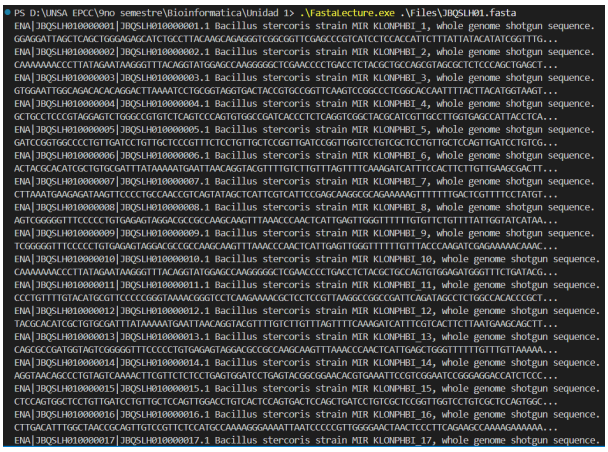

Figura 2: Mostrando el identificador y la secuencia.

2.3. Calcular la longitud mínima, máxima y promedio

Una vez que los datos del archivo **FASTA** han sido almacenados en una estructura adecuada, es posible realizar un análisis sobre las secuencias. En este caso, se busca calcular métricas básicas como la longitud mínima, máxima y el promedio de las secuencias.

1. La **longitud mínima** corresponde a la secuencia más corta dentro del conjunto de datos.

2. La **longitud máxima** corresponde a la secuencia más larga.

3. El **promedio** se obtiene sumando las longitudes de todas las secuencias y dividiendo entre el número total de registros.

Para almacenar estos resultados, se define una estructura que agrupa dichas métricas y con la ayuda de esa estructura si requerimos de más análisis podemos aumentarlo en la estructura.

```cpp
#define ll long long

struct FastaMetrics {
    ll maximum;
    ll minimum;
    double average;
};

FastaMetrics calculate_metrics(const std::vector<FastaRecordStructure>& records
    ) {
    FastaMetrics metrics_fasta = {0, LLONG_MAX, 0.0};
    ll total_length = 0;

    for (const auto &record : records) {
        ll sequence_length = record.sequence.length();
        total_length += sequence_length;

        if (sequence_length > metrics_fasta.maximum) {
            metrics_fasta.maximum = sequence_length;
        }

        if (sequence_length < metrics_fasta.minimum) {
            metrics_fasta.minimum = sequence_length;
        }
    }

    metrics_fasta.average = records.empty() ? 0.0 : static_cast<double>(
        total_length) / records.size();

    return metrics_fasta;
}
```

Entonces con esto vamos a obtener la secuencia más corta, la secuencia más larga y el promedio de todas las secuencias que hay en el archivo **FASTA** que para más adelante será necesario. En la Figura 3 se muestra.

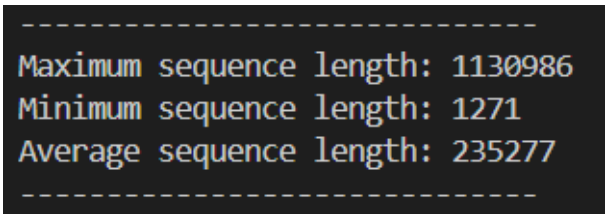

Figura 3: Calculando el mínimo, máximo y promedio.

2.4. Proporción GC de cada secuencia

Por último, vamos a calcular la **proporción de contenido GC** para cada secuencia, la cual representa la fracción de nucleótidos que corresponden a **Guanina (G)** y **Citosina (C)** respecto al total de la secuencia.

1. Se recorre cada secuencia carácter por carácter.
2. Se contabiliza la cantidad de nucleótidos **G** y **C**.
3. La proporción GC se calcula como el cociente entre la cantidad de **G** + **C** y la longitud total de la secuencia.

Matemáticamente, se define como:

$$GC = \frac{\#G + \#C}{\text{longitud de la secuencia}}$$

Este cálculo permite caracterizar las secuencias, ya que el contenido GC está relacionado con propiedades biológicas como la estabilidad del ADN. En código se vería de la siguiente manera:

```cpp
double count_proportion_gc(const std::string &sequence) {
    ll gc_count = 0;

    for (char gc : sequence) {
        if (gc == 'G' || gc == 'C' || gc == 'g' || gc == 'c') {
            gc_count++;
        }
    }

    double proportion_gc = sequence.empty() ? 0.0 : (static_cast<double>(
        gc_count) / sequence.length()) * 100.0;

    return proportion_gc;
}
```

Ahora con la operación que vamos a utilizar vamos a saber la proporción de **G** y **C** que hay en cada secuencia, la Figura 4 muestra esa cantidad.

```
Proportion GC per sequence:
ENA|JBQSLH010000001|JBQSLH010000001.1 Bacillus stercoris strain MIR KLONPHBI_1, whole genome shotgun sequence.: 42.02%
ENA|JBQSLH010000002|JBQSLH010000002.1 Bacillus stercoris strain MIR KLONPHBI_2, whole genome shotgun sequence.: 44.03%
ENA|JBQSLH010000003|JBQSLH010000003.1 Bacillus stercoris strain MIR KLONPHBI_3, whole genome shotgun sequence.: 43.49%
ENA|JBQSLH010000004|JBQSLH010000004.1 Bacillus stercoris strain MIR KLONPHBI_4, whole genome shotgun sequence.: 44.18%
ENA|JBQSLH010000005|JBQSLH010000005.1 Bacillus stercoris strain MIR KLONPHBI_5, whole genome shotgun sequence.: 45.12%
ENA|JBQSLH010000006|JBQSLH010000006.1 Bacillus stercoris strain MIR KLONPHBI_6, whole genome shotgun sequence.: 45.59%
ENA|JBQSLH010000007|JBQSLH010000007.1 Bacillus stercoris strain MIR KLONPHBI_7, whole genome shotgun sequence.: 37.24%
ENA|JBQSLH010000008|JBQSLH010000008.1 Bacillus stercoris strain MIR KLONPHBI_8, whole genome shotgun sequence.: 42.42%
ENA|JBQSLH010000009|JBQSLH010000009.1 Bacillus stercoris strain MIR KLONPHBI_9, whole genome shotgun sequence.: 42.67%
ENA|JBQSLH010000010|JBQSLH010000010.1 Bacillus stercoris strain MIR KLONPHBI_10, whole genome shotgun sequence.: 41.47%
ENA|JBQSLH010000011|JBQSLH010000011.1 Bacillus stercoris strain MIR KLONPHBI_11, whole genome shotgun sequence.: 46.10%
ENA|JBQSLH010000010|JBQSLH010000010.1 Bacillus stercoris strain MIR KLONPHBI_10, whole genome shotgun sequence.: 41.47%
ENA|JBQSLH010000011|JBQSLH010000011.1 Bacillus stercoris strain MIR KLONPHBI_11, whole genome shotgun sequence.: 46.10%
ENA|JBQSLH010000012|JBQSLH010000012.1 Bacillus stercoris strain MIR KLONPHBI_12, whole genome shotgun sequence.: 42.89%
ENA|JBQSLH010000013|JBQSLH010000013.1 Bacillus stercoris strain MIR KLONPHBI_13, whole genome shotgun sequence.: 41.81%
ENA|JBQSLH010000014|JBQSLH010000014.1 Bacillus stercoris strain MIR KLONPHBI_14, whole genome shotgun sequence.: 54.00%
ENA|JBQSLH010000015|JBQSLH010000015.1 Bacillus stercoris strain MIR KLONPHBI_15, whole genome shotgun sequence.: 58.27%
ENA|JBQSLH010000016|JBQSLH010000016.1 Bacillus stercoris strain MIR KLONPHBI_16, whole genome shotgun sequence.: 35.14%
ENA|JBQSLH010000017|JBQSLH010000017.1 Bacillus stercoris strain MIR KLONPHBI_17, whole genome shotgun sequence.: 53.95%
ENA|JBQSLH010000018|JBQSLH010000018.1 Bacillus stercoris strain MIR KLONPHBI_18, whole genome shotgun sequence.: 37.77%
PS D:\UNSA EPCC\9no semestre\Bioinformatica\Unidad 1>
```

Figura 4: Proporción GC en cada secuencia.

3. Programa completo

Ahora que tenemos los ejercicios completos, el código completo con cada sección sería el siguiente:

```cpp
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <iomanip>
```

```

#define ll long long

struct FastaRecordStructure {
    std::string identifier;
    std::string sequence;
};

struct FastaMetrics {
    ll maximum;
    ll minimum;
    double average;
};

void read_file_fasta(const std::string &file_fasta, std::vector<
    FastaRecordStructure> &records) {
    std::ifstream file(file_fasta);
    if (!file.is_open()) {
        std::cerr << "Error opening file: " << file_fasta << std::endl;
        return;
    }

    std::string line, sequence, header;
    while (std::getline(file, line)) {
        if (line.empty()) {
            continue;
        }

        if (line[0] == '>') {
            if (!header.empty()) {
                records.push_back({header, sequence});
                sequence.clear();
            }
            header = line.substr(1);
        }
        else {
            sequence += line;
        }
    }

    if (!header.empty()) {
        records.push_back({header, sequence});
    }

    file.close();
}

FastaMetrics calculate_metrics(const std::vector<FastaRecordStructure>& records
    ) {
    FastaMetrics metrics_fasta = {0, LLONG_MAX, 0.0};
    ll total_length = 0;

    for (const auto &record : records) {
        ll sequence_length = record.sequence.length();
        total_length += sequence_length;

        if (sequence_length > metrics_fasta.maximum) {
            metrics_fasta.maximum = sequence_length;
        }

        if (sequence_length < metrics_fasta.minimum) {
            metrics_fasta.minimum = sequence_length;
        }
```

```
    }

    metrics_fasta.average = records.empty() ? 0.0 : static_cast<double>(total_length) / records.size();

    return metrics_fasta;
}

double count_proportion_gc(const std::string &sequence) {
    ll gc_count = 0;

    for (char gc : sequence) {
        if (gc == 'G' || gc == 'C' || gc == 'g' || gc == 'c') {
            gc_count++;
        }
    }

    double proportion_gc = sequence.empty() ? 0.0 : (static_cast<double>(gc_count) / sequence.length()) * 100.0;

    return proportion_gc;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        std::cerr << "Usage: " << argv[0] << " <file.fasta>" << std::endl;
        return 1;
    }

    std::vector<FastaRecordStructure> records;
    std::string file_fasta = argv[1];
    read_file_fasta(file_fasta, records);

    FastaMetrics metrics = calculate_metrics(records);

    const int LIMIT = 100;

    for (const auto &record : records) {
        std::cout << record.identifier << "\n";
        if (record.sequence.length() > LIMIT) {
            std::cout << record.sequence.substr(0, LIMIT) << "...\n";
        } else {
            std::cout << record.sequence << "\n";
        }
    }

    std::cout << "------------------------------\n";
    std::cout << "Maximum sequence length: " << metrics.maximum << "\n";
    std::cout << "Minimum sequence length: " << metrics.minimum << "\n";
    std::cout << "Average sequence length: " << metrics.average << "\n";
    std::cout << "------------------------------\n";
    std::cout << "Proportion GC per sequence:\n";

    for (const auto &record : records) {
        std::cout << record.identifier << ": " << std::fixed << std::setprecision(2) << count_proportion_gc(record.sequence) << "%\n";
    }

    return 0;
}
```